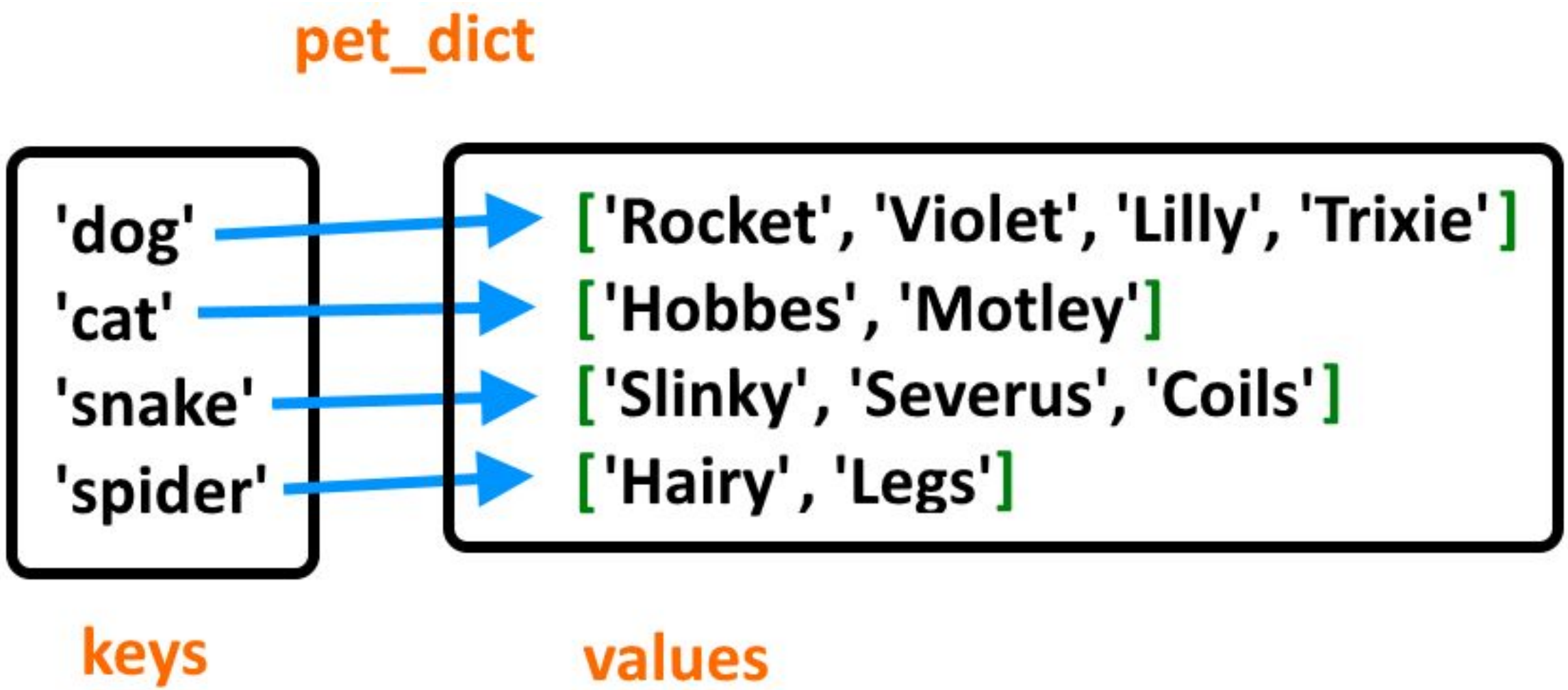
Traversing Dictionaries

ISTA 130

Dictionaries

Like a list, a dictionary allows us to store multiple values in a single container.

- With a list we access those values via index numbers
  - you can think of an index as the location of an element within a list
- With a dictionary we access the values via `keys`
  - you can think of a key as the location of an element within a dictionary


pet_dict
'dog'
'cat'
'snake'
'spider'
['Rocket', 'Violet', 'Lilly', 'Trixie']
['Hobbes', 'Motley']
['Slinky', 'Severus', 'Coils']
['Hairy', 'Legs']
keys
values

Some Code:

```
if pet_type in pd:
    pd[pet_type].append(pet_name)
else:
    pd[pet_type] = [pet_name]
```

Operator: `in`

- The **in** operator looks for **keys**, *not* values:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'
```

- So we can verify if a given *key* is in a dictionary:

```
print('Cabrales' in cheeses) → True
print('Spain' in cheeses) → False
print('Easy Cheese' in cheeses) → False
print('Easy Cheese' not in cheeses) → True
```

Function: `len()`

- The `len` function tells you the *number of key-value pairs* in a dictionary:

```
print(cheeses) →
{'Leyden': 'Netherlands', 'Manchego': 'Spain',
'Wensleydale': 'England', 'Cabrales': 'Spain'}

print(len(cheeses)) → 4

cheeses['Cheddar'] = 'England'

print(len(cheeses)) → 5
```

Methods: `values()`

- **values** method returns "an object providing a view on"[1] all values in a dictionary.

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'
```

- To check if a *value* is **in** a dictionary we use the `values` method:

```
print('Spain' in cheeses.values()) → True

print('Cabrales' in cheeses.values()) → False
```

Methods: `values()`

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'
```

- Sometimes it's useful to get a list containing a dictionary's values. We can convert the view object to a list like this:

```
countries = list(cheeses.values())
print(countries) →
['Spain', 'Netherlands', 'Spain']
```

Methods: `keys()`

- The **keys** method returns a view object containing all keys in a dictionary.

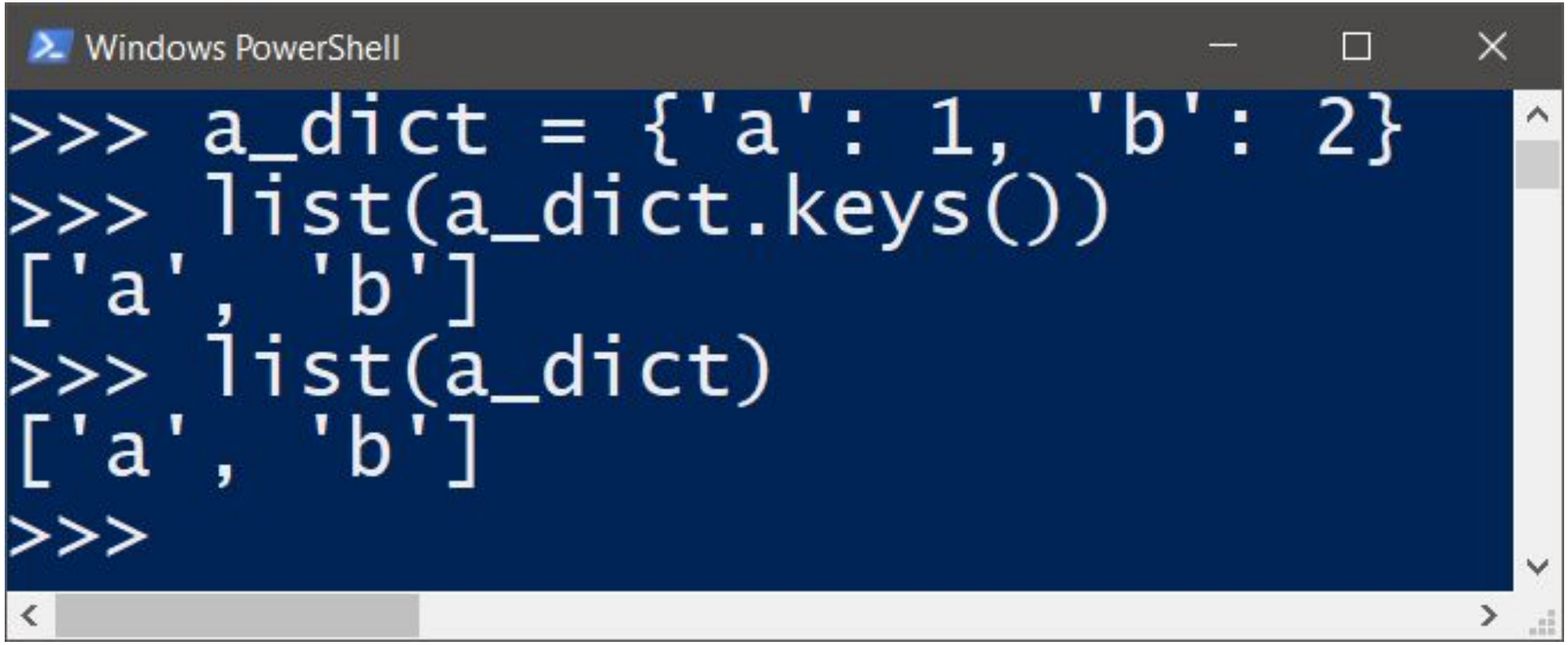
```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'
```

- We can convert the view object to a list like this:

```
cheese_names = list(cheeses.keys())
print(cheese_names) →
['Leyden', 'Cabrales', 'Manchego']
```

Methods: We Never Really Need

`keys()`



Methods: `pop()`

- The **pop** method allows you to remove a key-value pair from a dictionary:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'

print(cheeses) →
{'Leyden':  'Netherlands', 'Manchego':  'Spain'}

popped_cheese = cheeses.pop('Leyden')

print(popped_cheese) → Netherlands

print(cheeses) →
{'Manchego':  'Spain}
```

Copies: `dict`

- The `dict` function makes a copy of a dictionary:

```
cheeses = {}
cheeses["Manchego"] = "Spain"
cheeses["Leyden"] = "Netherlands"
cheeses["Cabrales"] = "Spain"

cheeses_copy = dict(cheeses)

cheeses["Manchego"] = "yummiest"

print(cheeses["Manchego"]) → yummiest

print(cheeses_copy["Manchego"]) → Spain
```

I like the `copy()` method

Methods: `clear()`

- The `clear` method removes all key-value pairs from a dictionary:

```
cheeses = {}
cheeses["Manchego"] = "Spain"
cheeses["Leyden"] = "Netherlands"
cheeses["Cabrales"] = "Spain"

cheeses.clear()

print(cheeses) → {}
```

`for` Loop on a Dictionary

A for-loop will iterate over the *keys* in a dictionary:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'

for cheese in cheeses:
    print(cheese)
```

- Output →

```
Leyden
Manchego
Cabrales
```

`for` Loop on a Dictionary

Since the for-loop's variable refers to a different key on each iteration, we can visit each of the values like this:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'

for cheese in cheeses:
    print(cheeses[cheese])
```

- Output →

```
Netherlands
Spain
Spain
```

`for` Loop on a Dictionary

If we want to print out everything in the dictionary:

```
cheeses = dict()
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'

print('Some yummy cheeses:')
for key in cheeses:
   print('\t' + key + ' ---> ' + cheeses[key])
```

- Output →

```
Some yummy cheeses:
   Leyden ---> Netherlands
   Manchego ---> Spain
   Cabrales ---> Spain
```

`for` Loop on a Dictionary

and since a dictionary is mutable we can do this:

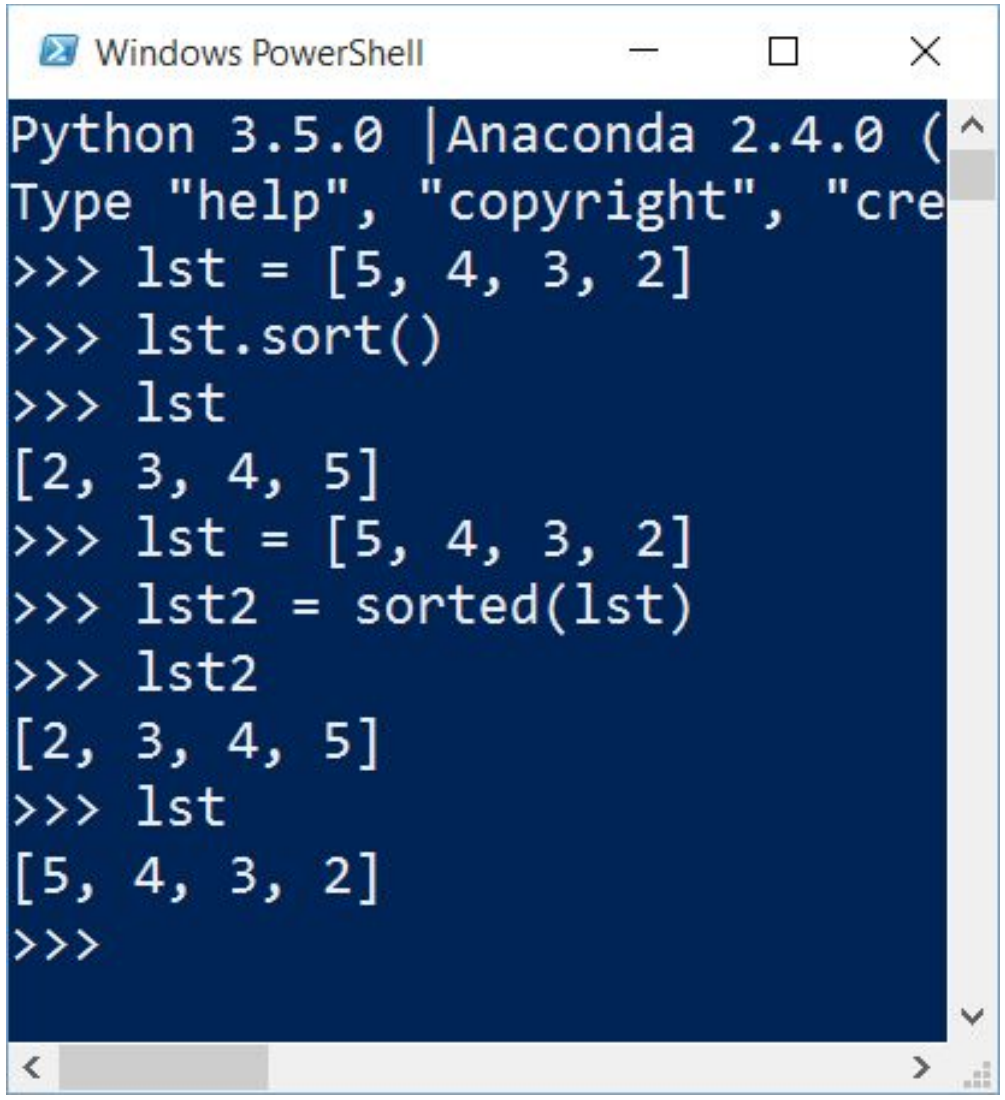
```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'

for key in cheeses:
   cheeses[key] = cheeses[key].upper()
print(cheeses)
```

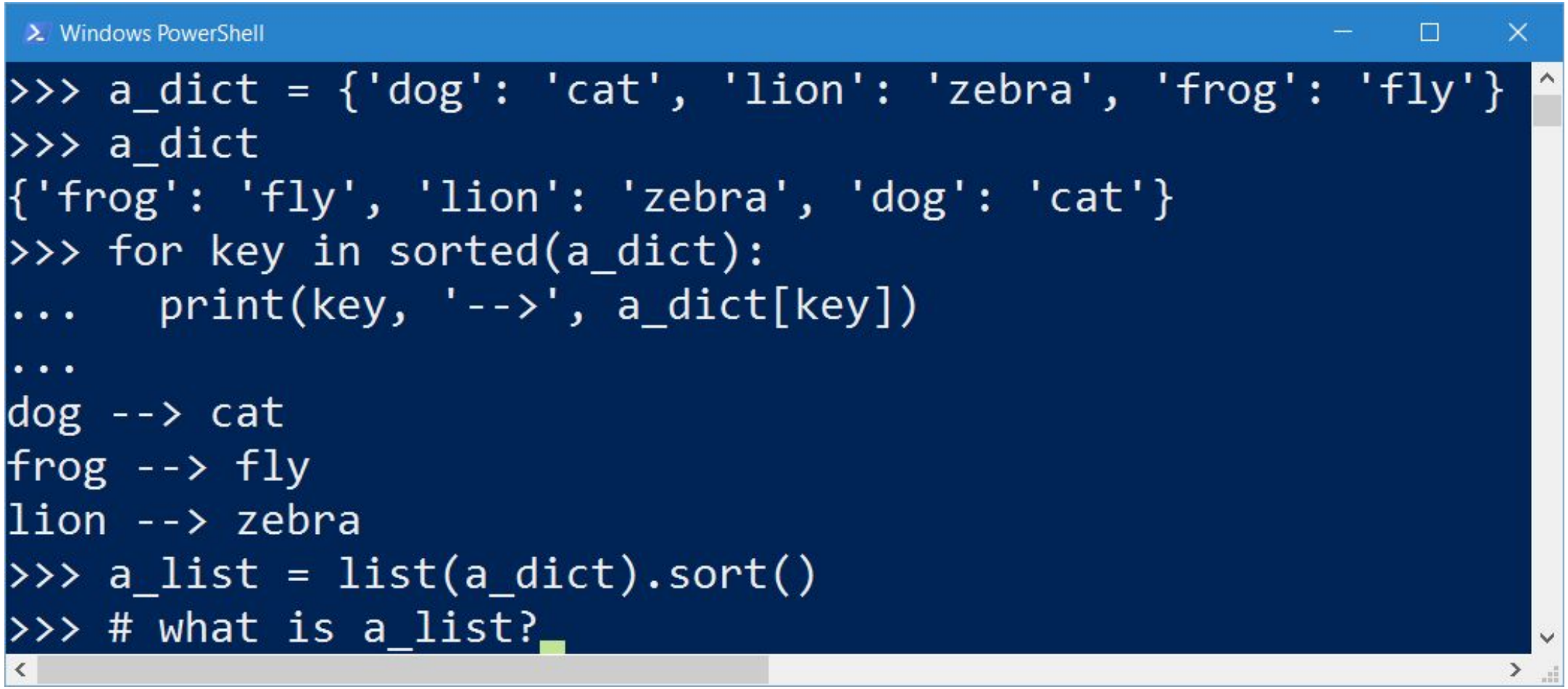
- Output →

```
{'Leyden':  'NETHERLANDS', 'Manchego':  'SPAIN',
'Cabrales':  'SPAIN'}
```

Back to Lists: The `sort` Method vs. the `sorted` Function



Traversing Dictionaries: Use `sorted()` not `sort()`



Example: Countries to Cheese

- In our current cheese example we have cheeses mapped to countries
  - How could we print out a list of all cheeses from a given country?
  - We could do it, but it would be awkward
- In this case it would make more sense to use countries as keys
  - Each key in a dictionary maps to a *single* value
    - we want each country to map to many cheeses
    - use a list as the value

Creating, Methods, Operators, Iterations

- `a_dict = {}` → creates a new dictionary object
- `a_dict = dict()` → creates a new dictionary object
- `a_dict.keys()` → returns a view of all keys
- `a_dict.values()` → returns view of all values
- **in** operator → test if a key is in a dictionary
- `a_dict[key]` → accesses the value for given key
  - Error if the key does not exist
- `a_dict.pop(key)` → remove key-value pair, return value
  - Error if the key does not exist
- `len(a_dict)` → return number of keys in dictionary
- `for key in a_dict:` → iterates over keys
- `a_dict.clear()` → remove everything from dictionary
- `a_copy = dict(a_dict)` → make a copy

```
for key in sorted(a_dict):
```

- Python coders often use this syntax.
- We can't always – what if the keys are not comparable to each other?
- In most real-life situations, this works fine.
- Do NOT alter mutable objects while iterating over them. It is a really bad idea.